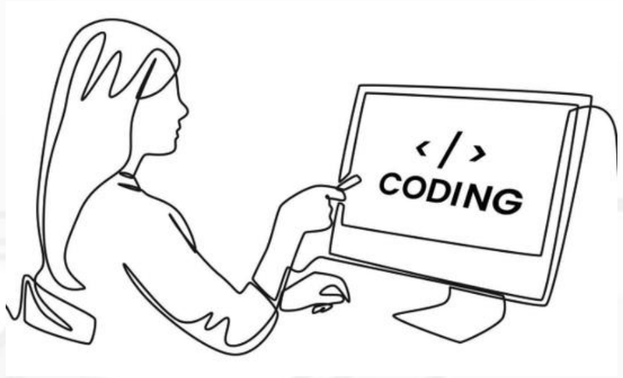
FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Funciones y Modularización

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano



Objetivos

- Punteros (repaso)
- Definición de funciones
- Reglas de Ámbito
- Variables locales y globales
- Parámetros
- Buenas prácticas



Herramientas

OnlineGDB es un entorno de desarrollo integrado (IDE) en la nube que combina edición de código, compilación y ejecución de programas en C/C++ con un solo clic:

https://www.onlinegdb.com/

Punteros (repaso)

¿ Qué es un puntero ?

- Un puntero es una variable especial que ***guarda direcciones de memoria***
- Se dice que "**apunta a**" dirección de memoria, que contiene un valor con un tipo específico de dato, definido previamente.

¿ Cómo se define ?

- tipo ***nombrePuntero;**
- Se utilizan los operadores de referencia (**&**) y desreferencia (***)
- Operador de referencia (**&**): obtiene la direccion de memoria de una variable
- Operador de desreferencia (***): acceder al valor que hay en esa dirección.

Ejemplo

```c
#include <stdio.h>

int main() {
    int x = 42;        // una variable normal
    int *p = &x;       // p guarda la dirección de x (uso &)

    printf("Valor de x: %d\n", x);
    printf("Direccion de x (&x): %p\n", &x);
    printf("Direccion guardada en p: %p\n", p);

    // Uso * para acceder al valor desde el puntero
    printf("Valor al que apunta p (*p): %d\n", *p);

    // Modifico el valor de x usando el puntero
    *p = 99;   // aquí uso * para desreferenciar y asignar un nuevo valor

    printf("Nuevo valor de x: %d\n", x);
    printf("Valor al que apunta p (*p): %d\n", *p);

    return 0;
}
```

```
Valor de x: 42
Direccion de x (&x): 0x7ffee00f4c1c
Direccion guardada en p: 0x7ffee00f4c1c
Valor al que apunta p (*p): 42
Nuevo valor de x: 99
Valor al que apunta p (*p): 99
```

Introducción

¿Por qué modularizar?

- Problema: un programa lineal crece y se vuelve inmanejable.
- Solución: divide y vencerás → separar en subprogramas.
- En C: todo son funciones (aunque algunas funcionan como procedimientos).
- Un solo bloque → difícil de mantener
- Dividir en funciones → claro, reutilizable, escalable

```c
#include <stdio.h>

// Prototipos
int sumar(int a, int b);
int restar(int a, int b);

int main() {
    int x = 5, y = 3;
    int resultado;

    resultado = sumar(x, y);
    mostrarResultado(resultado);

    resultado = restar(x, y);
    mostrarResultado(resultado);

    duplicarPorReferencia(&x);
    mostrarResultado(x);

    return 0;
}

// Definiciones
int sumar(int a, int b) {
    // return a + b;
}

int restar(int a, int b) {
    // return a - b;
}
```

Funciones

Concepto

- ***Una función devuelve un valor*** con **return**.
- Representa un cálculo o transformación.
- Facilita reutilización y claridad.

Estructura de declaración de función

tipo nombre_funcion(tipo1 param1, tipo2 param2);

En la **definición** de la funcion, hay que devolver el valor con **return**

```
int sumar(int a, int b) {
        return a + b;
    }
```

```c
#include <stdio.h>

// Función: devuelve un valor
int sumar(int a, int b) {
    return a + b;
}

int main() {
    int x = 5, y = 3;
    int resultado = sumar(x, y);

    printf("La suma es %d\n", resultado);
    return 0;
}

```

Procedimientos

Concepto

- ***Un procedimiento no devuelve valor*** (en C también es una función)
- Su objetivo es realizar una acción, no un cálculo.
- Se declara con void

Estructura de declaración de procedimiento

void nombre_proc(tipo1 param1, tipo2 param2);

Ejemplo:

void imprimirSuma(**int** a, **int** b);

```c
#include <stdio.h>

// Procedimiento: no devuelve nada
void mostrarMensaje(char nombre[]) {
    printf("Hola %s, bienvenido!\n", nombre);
}

int main() {
    mostrarMensaje("Gustavo");
    return 0;
}

```

Tipos de datos void

Concepto

- **void** es un tipo de dato especial que significa “sin valor” o “nada”.
- Se utiliza en dos contextos principales:
    - Funciones sin valor de retorno (procedimientos)
    - Funciones sin parámetros
    - Punteros genéricos (**void ***)
- Un puntero generico, puede apuntar cualquier tipo de dato

```c
//---------funcion sin valor de retorno------
void mostrarMensaje() {
    printf("Hola!\n");
}

//---------Funcion sin parametros -----------

int generarNumero(void) {
    return 42;
}

//---------Puntero generico -----------------

void *ptr;  // puntero genérico

```

Parámetros y Alcance de Variables

- **Variables locales**
  - Se declaran dentro de una función.
  - Solo son visibles en esa función.
  - Su tiempo de vida dura mientras la función se ejecuta.
- **Variables globales**
  - Se declaran fuera de las funciones (antes de main).
  - Son visibles y utilizables en todo el programa.
  - Se recomienda usarlas con cuidado para evitar errores y confusión.

```c
//   ====== Sección de librerías ======
#include <stdio.h>

//   ====== Declaración de constantes ======
//  #define PI 3.1415

//   ====== Declaración de variables globales ======

int contadorGlobal = 0;

//   ====== Prototipos de funciones ======
int sumar(int a, int b);

//   ====== Función principal (main) ======

int main(void) {
    // Declaración de variables locales
    int x, y, resultado;
    contadorGlobal = 1;  //modifico la variable global

    return 0; // devuelve 0 al sistema operativo -> ejecución correcta
}

//  ====== Implementación de funciones ======

int sumar(int a, int b) {
    return a + b;
}
```

Pasaje de parámetros

- **Por copia (valor)**
  - Se pasa una copia de la variable. Lo que cambie dentro de la función no afecta al original.
- **Por referencia (dirección)**
  - Se pasa un puntero, la función puede modificar la variable original

Modo	Como funciona	Efecto	Uso tipico
Por copia (valor)	Se envía una copia de la variable (más lento)	Los cambios dentro de la función no afectan al origina	Operaciones de solo lectura
Por referencia	Se envía la dirección de memoria (puntero -más rápido)	La función puede modificar el valor original	Cuando necesitamos modificar variables o devolver más de un resultado

```c
#include <stdio.h>

// Pasaje por copia (valor)
void duplicar(int x) {
    x = x * 2;
}

// Pasaje por referencia (dirección)
void duplicarRef(int *x) {
    *x = *x * 2;
}

int main() {
    int a = 5;

    duplicar(a);       // copia → no cambia
    printf("Por copia: a = %d\n", a);

    duplicarRef(&a);   // referencia → cambia el original
    printf("Por referencia: a = %d\n", a);

    return 0;
}
```

Pasaje de parámetros

Ejemplo: pasar un vector como parámetro y calcular el promedio

En C, cuando usamos el nombre de un vector (por ejemplo v), este se interpreta como la dirección de memoria donde comienza el arreglo, es decir, la dirección de su primer elemento

El nombre de un vector entonces es equivalente a un puntero al primer elemento.

- v → dirección de v[0]
- *v → contenido de v[0]

(página 21 - Manual de referencia)

```c
#include <stdio.h>
#define N 5   // tamaño del vector

// Función que recibe un vector por referencia y devuelve el promedio
float promedio(int v[], int n) {
    int i, suma = 0;
    for (i = 0; i < n; i++) {
        suma += v[i];
    }
    return (float)suma / n;
}

// Procedimiento para mostrar el vector
void mostrar(int v[], int n) {
    int i;
    printf("Vector: ");
    for (i = 0; i < n; i++) {
        printf("%d ", v[i]);
    }
    printf("\n");
}

int main() {
    int vec[N] = {10, 20, 30, 40, 50};  // vector de ejemplo

    mostrar(vec, N); // paso el vector a un procedimiento
    printf("El promedio es: %.2f\n", promedio(vec, N));

    return 0;
}
```

Funciones Recursivas

¿Qué es la recursividad?

- Una función se llama a sí misma dentro de su definición.
- Se utiliza para problemas que pueden descomponerse en subproblemas más pequeños.
- Siempre debe haber una condición de corte para evitar bucles infinitos.

➢ factorial(5) → 5 * factorial(4) = 120

➢ factorial(4) → 4 * factorial(3) = 24

➢ factorial(3) → 3 * factorial(2) = 6

➢ factorial(2) → 2 * factorial(1) = 2

➢ factorial(1) → 1 * factorial(0) = 1

➢ factorial(0) devuelve 1 (condición base).

➢ **Resultado**: 5 * 4 * 3 * 2 * 1 = 120

```c
#include <stdio.h>

// Función recursiva
int factorial(int n) {
    if (n == 0)      // caso base
        return 1;
    else
        return n * factorial(n - 1); // llamada recursiva
}

int main() {
    int num = 5;
    printf("El factorial de %d es %d\n", num, factorial(num));
    return 0;
}
```

Que aprendimos hasta acá ?

- Que es modularizar? Como dividir el problema
- Funciones -> devuelve un valor
- Procedimientos -> no devuelve valor
- Nuevo tipo de datos void (sin valor)
- Alcance de variables
- Pasaje de parámetros por valor y por referencia

  Hasta aca C - en 3 semanas!



Próximo paso

Comparativa entre lenguajes clásicos

Lenguajes: C, Pascal, QuickBasic, COBOL, Assembler

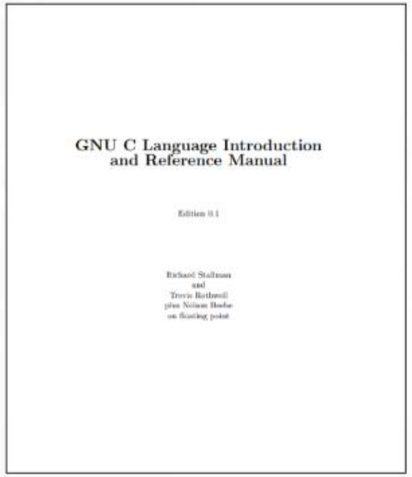
Bibliografía

- El lenguaje C: Manual de Referencia (Balestrini - Di Stefano)
- Programación en C para principiantes (Urrutia - Landa)
- Introducción a la programación en C (Marzal - Gracia)
- GNU C Language Introduction and Reference Manual (Richard Stallman)









FADENA
FACULTAD DE DEFENSA NACIONAL

¿Preguntas o dudas?



FADENA
FACULTAD DE DEFENSA NACIONAL